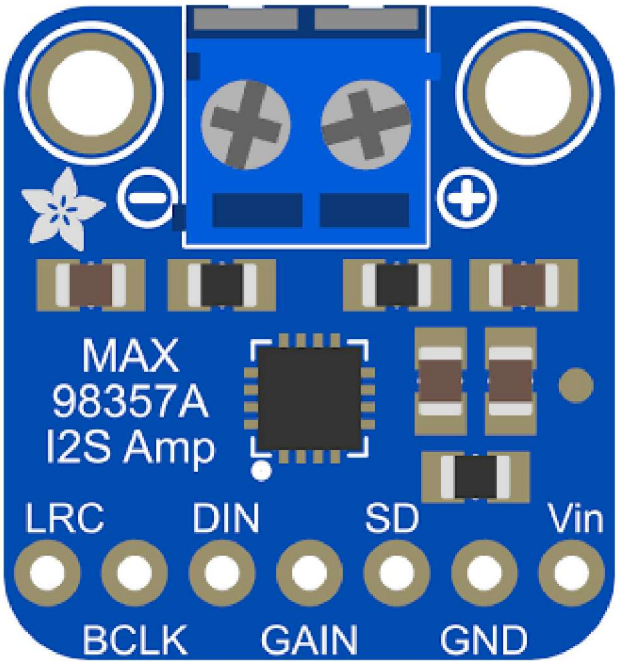
Tengo un chip MAX98357A I2S y quiero conectarlo a una raspberry pi pico 2w para hacer pruebas. Considera que estoy programando en C++ en VSCODE

El amplificador clase D **MAX98357A** se comunica mediante protocolo de audio digital **I2S**, el cual requiere tres líneas de reloj y datos más la alimentación. En el RP2350 (Raspberry Pi Pico 2 W), el protocolo I2S se implementa mediante la máquina de estados programable **PIO** o la librería `pico_audio_i2s` oficial del SDK.



Pines del módulo MAX98357A. Fuente: Cirkit Designer Docs

Esquema de Conexión

Pin MAX98357A	Pin Raspberry Pi Pico 2 W	Función / Descripción
VIN	**VBUS** (Pin 40) o **3V3** (Pin 36)	Para máxima potencia (hasta 3.2W en 4Ω) usa 5V de VBUS. Acepta 2.5V a 5.5V.
GND	**GND** (ej. Pin 38 o Pin 3)	Tierra común.

Pin MAX98357A	Pin Raspberry Pi Pico 2 W	Función / Descripción
BCLK (Bit Clock)	**GPIO 26** (Pin 31)	Reloj de bit continuo para sincronización de bits de audio.
LRC (Word Select / WS)	**GPIO 27** (Pin 32)	Reloj de cuadro (Left/Right Clock): define canal izquierdo o derecho.
DIN (Data In)	**GPIO 28** (Pin 34)	Línea de datos de audio digital multiplexada.
GAIN	*Sin conectar* (Flotante)	Por defecto deja una ganancia fija de **+9 dB** (adecuada para pruebas).
SD (Shutdown / Mode)	*Sin conectar* o Pull-up	Sin conectar mezcla $(L + R)/2$ a mono. Conectar con resistencia a 3.3V selecciona canal izquierdo o derecho.

Nota: *Puedes elegir cualquier conjunto de GPIOs contiguos en la Pico para el PIO, pero mantener* **Data (DIN), BCLK y LRC** *en pines ordenados simplifica el mapeo del periférico PIO.*

Configuración de Software (Pico SDK en C++)

El SDK oficial incluye el paquete `pico_audio` y `pico_audio_i2s` optimizado con DMA y PIO para transmisión sin bloqueo de CPU.

1. Modificación de `CMakeLists.txt`

Asegúrate de enlazar la biblioteca de audio I2S:

CMake

```
target_link_libraries(tu_proyecto
    pico_stdlib
```

```
    pico_audio_i2s
)
```

2. Código base en C++ (main.cpp)

C++

```cpp
#include "pico/stdlib.h"
#include "pico/audio_i2s.h"
#include <cmath>

#define PICO_AUDIO_I2S_DATA_PIN 28
#define PICO_AUDIO_I2S_CLOCK_PIN_BASE 26 // BCLK en 26, LRC en 27

const uint32_t SAMPLE_RATE = 44100;
const uint16_t SAMPLES_PER_BUFFER = 256;

int main() {
    stdio_init_all();

    // 1. Definir formato de audio estéreo 16-bit
    static audio_format_t audio_format = {
        .sample_freq = SAMPLE_RATE,
        .format = AUDIO_BUFFER_FORMAT_PCM_S16,
        .channel_count = 2,
    };

    static audio_buffer_format_t producer_format = {
        .format = &audio_format,
        .sample_stride = 4 // 2 canales * 2 bytes
    };

    // 2. Configurar pines y parámetros I2S
    struct audio_i2s_config config = {
        .data_pin = PICO_AUDIO_I2S_DATA_PIN,
        .clock_pin_base = PICO_AUDIO_I2S_CLOCK_PIN_BASE,
        .dma_channel = 0,
        .pio_sm = 0,
    };

    // 3. Inicializar buffer pool y salida I2S
    struct audio_buffer_pool *producer_pool = audio_new_producer_pool(
        &producer_format, 3, SAMPLES_PER_BUFFER);

    bool __unused ok = audio_i2s_setup(&audio_format, &config);
```

```
    audio_i2s_connect(producer_pool);
    audio_i2s_set_enabled(true);

    // 4. Bucle generador de tono de prueba (onda senoidal a 440 Hz)
    float phase = 0.0f;
    const float phase_inc = 2.0f * M_PI * 440.0f / SAMPLE_RATE;

    while (true) {
        struct audio_buffer *buffer = take_audio_buffer(producer_pool, true
        int16_t *samples = (int16_t *)buffer->buffer->bytes;

        for (uint16_t i = 0; i < SAMPLES_PER_BUFFER; i++) {
            int16_t val = (int16_t)(sinf(phase) * 8000.0f); // Volumen mode
            samples[i * 2] = val;     // Canal Izquierdo
            samples[i * 2 + 1] = val; // Canal Derecho
            phase += phase_inc;
            if (phase >= 2.0f * M_PI) phase -= 2.0f * M_PI;
        }

        buffer->sample_count = SAMPLES_PER_BUFFER;
        give_audio_buffer(producer_pool, buffer);
    }

    return 0;
}
```

¿Quieres que preparemos la configuración de CMake o un generador de síntesis por tablas (wavetable)?

↪ Sí

Como deberia conectar el max98357a a la raspi y otros elementos

Para conectar el **MAX98357A** a la Raspberry Pi Pico 2 W y a los demás componentes del circuito (alimentación, parlante y control de ganancia/canal), el conexionado se divide en tres secciones: **interfaz I2S con la Pico**, **salida al parlante** y **configuración de pines de control**.

1. Conexión I2S y Alimentación (MAX98357A ↔ Pico 2 W)

Pin MAX98357A	Pin en Pico 2 W	Tipo de Pin	Descripción
VIN	**Pin 40 (VBUS)** o **Pin 36 (3V3)**	Alimentación	**Recomenda (5V):** permite hasta 3.2W e usas 3.3V, la máxima se re ~0.8W.
GND	**Pin 38, 3 o 28 (GND)**	Tierra	Debe compa común con l
BCLK	**GPIO 26 (Pin 31)**	Reloj de Bit	Bit Clock par transferenci
LRC	**GPIO 27 (Pin 32)**	Word Select (WS)	Left/Right Cl (cambia cad de audio).
DIN	**GPIO 28 (Pin 34)**	Datos	Data In con l muestras PC multiplexada

2. Conexión al Parlante (Speaker)

El MAX98357A tiene una salida de amplificación diferencial **BTL (Bridge-Tied Load)**, no una salida single-ended común:

+ (Speaker +): Terminal positivo del parlante.

- (Speaker -): Terminal negativo del parlante.

Especificaciones recomendadas del parlante:

Impedancia: **4 Ω u 8 Ω**.

Potencia nominal: de **2 W a 5 W**.

⚠️ **Atención: Nunca conectes el pin - del parlante a GND.** La salida es balanceada; poner cualquiera de los dos polos a tierra dañará permanentemente el circuito integrado.

3. Configuración de Modo y Ganancia (Pines SD y GAIN)

Ambos pines incluyen resistencias internas pull-down/pull-up para funcionar por defecto sin necesidad de cables adicionales si se busca la configuración estándar:

Pin SD (Shutdown / Channel Select)

Define si el amplificador entra en bajo consumo o qué canal reproduce (la señal I2S es estéreo pero el chip es mono):

Flotante (desconectado): Modo estéreo promediado: mezcla el canal izquierdo y derecho $(L + R)/2$. *(Recomendado para pruebas simples)*.

Pull-up a 3.3V (resistencia de 10kΩ a 3V3): Reproduce únicamente el canal derecho (**Right**).

Resistencia de 100kΩ a 3.3V: Reproduce únicamente el canal izquierdo (**Left**).

A GND: Apaga el amplificador (Shutdown mode, consumo de corriente < 1 µA).

Pin GAIN (Ganancia)

Ajusta la ganancia interna de amplificación según la impedancia de tu parlante y el nivel de señal de entrada:

Conexión del pin GAIN	Ganancia resultante
Desconectado (Flotante)	**+9 dB** *(Valor por defecto recomendado)*
Conectado a GND	**+3 dB**
Conectado a VIN	**+12 dB**
Resistencia 100kΩ a GND	**+6 dB**
Resistencia 100kΩ a VIN	**+15 dB**

4. Elementos Adicionales Recomendados (Filtros y Desacople)

Para asegurar estabilidad y evitar ruido o reinicios por picos de corriente:

1. **Condensador de desacople en VIN:** Coloca un capacitor electrolítico o de tantalio de **100 µF a 470 µF** en paralelo con un cerámico de **0.1 µF (100nF)** entre **VIN** y **GND**

del MAX98357A, lo más cerca posible de la placa. Los amplificadores clase D demandan picos de corriente abruptos al conmutar.

2. **Potenciómetro para control de volumen (Opcional):** Como el chip es un amplificador de ganancia fija/discreta, el volumen se controla preferentemente:

   Por software: Atenuando las muestras de audio PCM antes de enviarlas al DMA/PIO.

   Por hardware: Añadiendo un potenciómetro analógico en un pin ADC de la Pico (ej. GPIO 26/ADC0 si usas otros pines para I2S, o GPIO 29/ADC3) para leerlo y multiplicar el búfer de muestras por un factor de escala.

¿Quieres ver cómo escalar el volumen por software en C++ o configurar el

realiza un código que permita probar de manera simple el funcionamiento

Aquí tienes un proyecto mínimo y autocontenido para probar el **MAX98357A** generando un tono continuo (*beep* senoidal de 440 Hz) usando la librería `pico_audio_i2s` del SDK oficial de Raspberry Pi.

1. `CMakeLists.txt`

Crea o reemplaza tu archivo `CMakeLists.txt` con la configuración necesaria para habilitar la salida de audio I2S por PIO:

CMake

```
cmake_minimum_required(VERSION 3.13)

# Inicializar SDK de Raspberry Pi Pico
include(pico_sdk_import.cmake)

project(test_max98357a C CXX ASM)
set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)

pico_sdk_init()
```

```
add_executable(test_max98357a
    main.cpp
)

# Enlazar la librería de audio I2S y bibliotecas estándar
target_link_libraries(test_max98357a
    pico_stdlib
    pico_audio_i2s
    hardware_dma
    hardware_pio
)

# Habilitar salida USB/UART para depuración en serie
pico_enable_stdio_usb(test_max98357a 1)
pico_enable_stdio_uart(test_max98357a 0)

# Generar archivos .uf2, .bin, .hex
pico_add_extra_outputs(test_max98357a)
```

2. main.cpp

Este código configura el bus I2S a **44.1 kHz, 16 bits estéreo** en los pines:

BCLK: GPIO 26

LRC (WS): GPIO 27

DIN: GPIO 28

C++

```
#include <stdio.h>
#include <math.h>
#include "pico/stdlib.h"
#include "pico/audio_i2s.h"

// Pines asignados
#define PIN_I2S_DATA        28 // DIN
#define PIN_I2S_CLOCK_BASE  26 // BCLK = 26, LRC = 27 (la librería toma bas

// Parámetros de audio
#define SAMPLE_RATE         44100
#define BUFFER_SAMPLES      256
#define TONE_FREQ           440.0f // Frecuencia de la nota (La / A4)
```

```
#define AMPLITUDE           6000   // Rango de 0 a 32767 (6000 es un volume

int main() {
    stdio_init_all();
    sleep_ms(2000); // Pausa inicial para conectar el monitor serial si es
    printf("Iniciando prueba de audio I2S con MAX98357A...\n");

    // 1. Especificar formato de audio (PCM con signo, 16 bits, estéreo, 44
    static audio_format_t audio_format = {
        .sample_freq = SAMPLE_RATE,
        .format = AUDIO_BUFFER_FORMAT_PCM_S16,
        .channel_count = 2,
    };

    static audio_buffer_format_t producer_format = {
        .format = &audio_format,
        .sample_stride = 4 // 2 canales x 2 bytes (16 bits)
    };

    // 2. Configurar hardware de la máquina de estados PIO
    struct audio_i2s_config config = {
        .data_pin = PIN_I2S_DATA,
        .clock_pin_base = PIN_I2S_CLOCK_BASE,
        .dma_channel = 0,
        .pio_sm = 0,
    };

    // 3. Crear el pool de buffers (3 buffers de 256 muestras para evitar c
    struct audio_buffer_pool *producer_pool = audio_new_producer_pool(
        &producer_format,
        3,
        BUFFER_SAMPLES
    );

    // 4. Inicializar periférico I2S y habilitar transmisión
    audio_i2s_setup(&audio_format, &config);
    audio_i2s_connect(producer_pool);
    audio_i2s_set_enabled(true);

    printf("Audio activo. Generando tono continuo de %.1f Hz...\n", TONE_FR

    // Variables de síntesis senoidal
    float phase = 0.0f;
    const float phase_step = (2.0f * (float)M_PI * TONE_FREQ) / (float)SAMF

    while (true) {
        // Solicita un buffer disponible del pool (bloqueante)
        struct audio_buffer *buffer = take_audio_buffer(producer_pool, true
        int16_t *samples = (int16_t *)buffer->buffer->bytes;
```

```
            // Llenar el buffer con muestras senoidales
            for (uint16_t i = 0; i < BUFFER_SAMPLES; i++) {
                int16_t sample_val = (int16_t)(sinf(phase) * (float)AMPLITUDE);

                // Muestra en canal izquierdo y derecho
                samples[i * 2]     = sample_val; // Canal L
                samples[i * 2 + 1] = sample_val; // Canal R

                phase += phase_step;
                if (phase >= 2.0f * (float)M_PI) {
                    phase -= 2.0f * (float)M_PI;
                }
            }

            buffer->sample_count = BUFFER_SAMPLES;
            // Entregar el buffer al sistema DMA para reproducción
            give_audio_buffer(producer_pool, buffer);
        }

        return 0;
    }
```

Pasos para compilar en VS Code

1. Abre la carpeta del proyecto en VS Code con la extensión oficial **Raspberry Pi Pico**.
2. Verifica que el compilador esté configurado para la arquitectura de la **Pico 2 W** (`arm-none-eabi`).
3. Ejecuta **Compile** o haz clic en **Build** en la barra de estado inferior.
4. Conecta la Pico 2 W manteniendo presionado el botón **BOOTSEL** y arrastra el archivo `.uf2` generado (en la carpeta `build/`) a la unidad de almacenamiento.
5. Inmediatamente deberías escuchar un tono puro continuo y claro a través del parlante.

¿Quieres agregar un barrido de frecuencias (chirp) o reproducir notas musicales definidas en un array?

↪ Sí

El código debe hacer sonar la canción
Pokémon Red, Blue & Yellow - Gym Leader Battle Music

Para recrear el tema de combate de Líder de Gimnasio (**Gym Leader Battle Theme** de Junichi Masuda en *Pokémon Red/Blue/Yellow*), el enfoque ideal es usar una **onda de pulso / cuadrada**, el timbre nativo del chip de sonido de la Game Boy (canales de pulso CH1 y CH2).

El tema original corre a un tempo rápido de aproximadamente **185 BPM** en clave de Mi menor. El siguiente código implementa el icónico riff de introducción cromática seguido de las frases principales del tema A.

Código en C++ (`main.cpp`)

Reemplaza tu archivo `main.cpp` con el siguiente sintetizador de ondas de pulso monofónico/chiptune que lee una partitura estructurada:

C++

```cpp
#include <stdio.h>
#include <stdint.h>
#include "pico/stdlib.h"
#include "pico/audio_i2s.h"

// Pines I2S
#define PIN_I2S_DATA        28 // DIN
#define PIN_I2S_CLOCK_BASE  26 // BCLK = 26, LRC = 27

#define SAMPLE_RATE         44100
#define BUFFER_SAMPLES      256
#define VOLUME              5000 // Nivel de amplitud seguro para pruebas

// Frecuencias de notas musicales (en Hz)
#define REST   0.0f
#define D3     146.83f
#define DS3    155.56f
#define E3     164.81f
#define F3     174.61f
#define FS3    185.00f
#define G3     196.00f
#define GS3    207.65f
#define A3     220.00f
```

```
#define AS3     233.08f
#define B3      246.94f
#define C4      261.63f
#define CS4     277.18f
#define D4      293.66f
#define DS4     311.13f
#define E4      329.63f
#define F4      349.23f
#define FS4     369.99f
#define G4      392.00f
#define GS4     415.30f
#define A4      440.00f
#define AS4     466.16f
#define B4      493.88f
#define C5      523.25f
#define CS5     554.37f
#define D5      587.33f
#define DS5     622.25f
#define E5      659.25f

// Duraciones relativas (a ~185 BPM, una semicorchea dura aprox. 81 ms)
#define T_16TH   81
#define T_8TH    162
#define T_QTR    324
#define T_HALF   648

struct Note {
    float freq;
    uint32_t duration_ms;
};

// Transcripción de la Intro y Tema Principal de Gym Leader Battle (Pokémor
const Note melody[] = {
    // --- INTRO: Riff cromático descendente y ascendente clásico ---
    {E4, T_16TH}, {DS4, T_16TH}, {D4, T_16TH}, {CS4, T_16TH},
    {C4, T_16TH}, {B3, T_16TH},  {AS3, T_16TH}, {A3, T_16TH},
    {GS3, T_16TH}, {G3, T_16TH}, {FS3, T_16TH}, {F3, T_16TH},
    {E3, T_8TH},  {E3, T_16TH},  {G3, T_16TH},  {AS3, T_8TH},

    {E4, T_16TH}, {DS4, T_16TH}, {D4, T_16TH}, {CS4, T_16TH},
    {C4, T_16TH}, {B3, T_16TH},  {AS3, T_16TH}, {A3, T_16TH},
    {GS3, T_16TH}, {G3, T_16TH}, {FS3, T_16TH}, {F3, T_16TH},
    {B3, T_8TH},  {AS3, T_8TH},  {G3, T_8TH},   {E3, T_8TH},

    // Subida tensa previa a la melodía
    {E3, T_16TH}, {G3, T_16TH}, {B3, T_16TH}, {E4, T_16TH},
    {G4, T_16TH}, {E4, T_16TH}, {B3, T_16TH}, {G3, T_16TH},
    {DS4, T_16TH}, {FS4, T_16TH}, {B4, T_8TH}, {REST, T_16TH},
```

```
    // --- TEMA A (Melodía de combate) ---
    {E4, T_8TH}, {B3, T_16TH}, {E4, T_16TH}, {G4, T_8TH}, {FS4, T_8TH},
    {E4, T_8TH}, {D4, T_16TH}, {E4, T_16TH}, {B3, T_QTR},

    {E4, T_8TH}, {B3, T_16TH}, {E4, T_16TH}, {G4, T_8TH}, {A4, T_8TH},
    {B4, T_8TH}, {A4, T_8TH},  {G4, T_8TH},  {FS4, T_8TH},

    {E4, T_8TH}, {B3, T_16TH}, {E4, T_16TH}, {G4, T_8TH}, {FS4, T_8TH},
    {E4, T_8TH}, {G4, T_16TH}, {B4, T_16TH}, {D5, T_QTR},

    {C5, T_8TH}, {B4, T_8TH},  {A4, T_8TH},  {G4, T_8TH},
    {FS4, T_8TH}, {E4, T_8TH}, {DS4, T_QTR},

    // Cadencia rítmica
    {E4, T_16TH}, {E4, T_16TH}, {REST, T_16TH}, {E4, T_16TH},
    {G4, T_8TH},  {FS4, T_8TH}, {E4, T_QTR},
    {REST, T_QTR}
};

const size_t TOTAL_NOTES = sizeof(melody) / sizeof(Note);

int main() {
    stdio_init_all();

    static audio_format_t audio_format = {
        .sample_freq = SAMPLE_RATE,
        .format = AUDIO_BUFFER_FORMAT_PCM_S16,
        .channel_count = 2,
    };

    static audio_buffer_format_t producer_format = {
        .format = &audio_format,
        .sample_stride = 4
    };

    struct audio_i2s_config config = {
        .data_pin = PIN_I2S_DATA,
        .clock_pin_base = PIN_I2S_CLOCK_BASE,
        .dma_channel = 0,
        .pio_sm = 0,
    };

    struct audio_buffer_pool *producer_pool = audio_new_producer_pool(
        &producer_format, 3, BUFFER_SAMPLES);

    audio_i2s_setup(&audio_format, &config);
    audio_i2s_connect(producer_pool);
    audio_i2s_set_enabled(true);
```

```
    float phase = 0.0f;
    size_t note_idx = 0;
    uint32_t samples_left_in_note = 0;

    while (true) {
        struct audio_buffer *buffer = take_audio_buffer(producer_pool, true
        int16_t *samples = (int16_t *)buffer->buffer->bytes;

        for (uint16_t i = 0; i < BUFFER_SAMPLES; i++) {
            // Avanzar a la siguiente nota si la actual terminó
            if (samples_left_in_note == 0) {
                note_idx = (note_idx + 1) % TOTAL_NOTES;
                samples_left_in_note = (melody[note_idx].duration_ms * SAMP
            }

            float current_freq = melody[note_idx].freq;
            int16_t sample_val = 0;

            if (current_freq > 0.0f) {
                // Síntesis de onda de pulso (duty cycle 50% estilo Game Bo
                sample_val = (phase < 0.5f) ? VOLUME : -VOLUME;

                // Avanzar fase de 0.0 a 1.0
                phase += current_freq / (float)SAMPLE_RATE;
                if (phase >= 1.0f) phase -= 1.0f;
            } else {
                // Silencio
                sample_val = 0;
                phase = 0.0f;
            }

            samples[i * 2]     = sample_val;
            samples[i * 2 + 1] = sample_val;

            samples_left_in_note--;
        }

        buffer->sample_count = BUFFER_SAMPLES;
        give_audio_buffer(producer_pool, buffer);
    }

    return 0;
}
```

Detalles del Funcionamiento

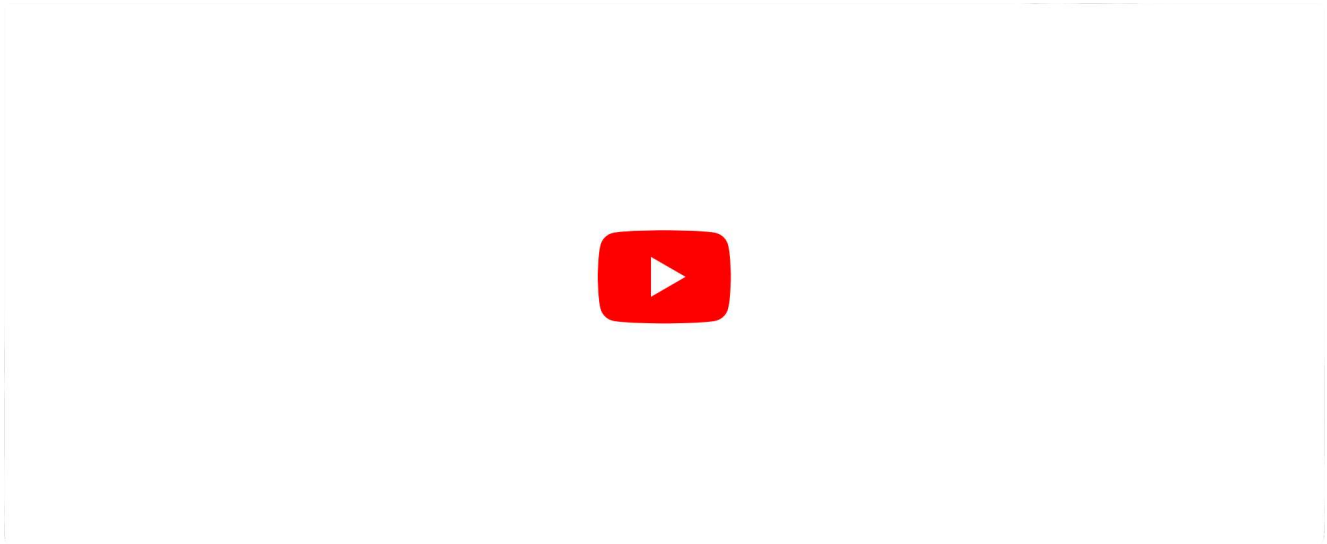
1. **Timbre Game Boy (Square Wave):** En lugar de usar `sinf()`, el oscilador alterna entre `+VOLUME` y `-VOLUME` según si la fase acumulada está por debajo de `0.5f`. Esto produce el timbre digital crujiente característico del chip de sonido DMG-01 de la Game Boy original.
2. **Tempo estricto:** La base de tiempo usa `T_16TH` (~81 ms por semicorchea) para reflejar los ~185 BPM de batalla rápida del juego. Project Nayuki
3. **Bucle continuo:** Al agotar el arreglo de notas, el índice vuelve a 0 automáticamente con el operador `% TOTAL_NOTES`, repitiendo el tema en bucle ininterrumpido a través de la cola DMA del PIO.

Si quieres escuchar la versión original con todos los canales sonando en simultáneo para contrastar los timbres de pulso, puedes revisar este video del tema de combate contra líder de gimnasio en Pokémon Blue/Red. Esta referencia te servirá para comparar el tempo de 185 BPM y los cambios rítmicos de la introducción original.

¿Quieres agregar polifonía en el segundo núcleo de la Pico para sumar la línea de bajo?

↪ Sí

Pokemon Blue/Red - Gym Leader Battle!
pokemonmusicmaster · 785 mil visualizaciones



Ampli Ampli

Quiero hacer una clase para poder utilizar el MAX98357A, considerando que se utilizaran archivos WAV que se transforman en hexadecimales.

Para diseñar la arquitectura de la clase `Ampli` de forma robusta, adaptada a la Raspberry Pi Pico 2 W y a tus archivos WAV en hexadecimal, necesito aclarar los siguientes puntos:

1. **Estructura de los datos hexadecimales:**

   ¿Los arreglos hexadecimales incluirán el **encabezado WAV completo** (los 44 bytes de cabecera `RIFF` / `WAVE`), requiriendo que la clase parsee dinámicamente frecuencia, canales y tamaño?

   ¿O exportarás los datos como **muestras PCM brutas (raw samples)**, por ejemplo arrays `const int16_t[]` o `const uint8_t[]` con formato fijo (como 16 bits mono/estéreo a 11025 Hz o 44100 Hz)?

2. **Modelo de ejecución (Bloqueante vs. No bloqueante):**

   ¿`reproducir()` debe detener la ejecución de todo el programa hasta que el sonido termine (bloqueante con un bucle `while`), o debe ser **asíncrona**?

   Si es asíncrona, ¿prefieres que el llenado del buffer se gestione mediante una función `actualizar()` / `update()` llamada en el `while(true)` principal, usando alarmas/interrupciones por software, o delegando el streaming a un hilo en el **segundo núcleo (Core 1)** de la Pico?

3. **Almacenamiento y registro de sonidos:**

   ¿Cómo imaginas enlazar la lista de sonidos a `sonidoActual`? ¿Registrando punteros a los arrays en la clase (por ejemplo, un array de punteros/estructuras `Sound { const uint8_t* data; size_t length; }`), o pasándole el buffer directamente al método?

4. **Entrada para el control de volumen:**

   Al incluir `hardware/adc.h` en tu cabecera, ¿el método `controlVolumen()` debe leer periódicamente un pin analógico (potenciómetro vía ADC) y mapear ese valor a 0–100, o debe recibir un parámetro numérico `void controlVolumen(int nuevoVolumen)` fijado por código?

1. se utilizarán arrays, en formato mono a 11025hz
2. reproducir no debe afectar ni detener otras funciones como lo son `controlVolumen()`. Además, no debe llamarse al método nuevamente si el sonido que se está

Gracias por aclarar los puntos 1, 2 y 4. Con eso definido:

Los datos serán arreglos de muestras PCM mono a **11025 Hz**.

La reproducción será **no bloqueante / asíncrona** para no congelar la ejecución ni la lectura de variables, e ignorará llamadas redundantes si un sonido ya está en curso. H

El volumen tomará la variable `volumen` (0 a 100) y aplicará una curva de atenuación **logarítmica**. H

Explicación detallada del Punto 3 (Almacenamiento y registro de sonidos)

En tu archivo de cabecera colocaste la variable:

C++

```
int sonidoActual = -1; // variable que enlaza al listado de sonidos que se
```

Para que `reproducir()` sepa qué reproducir cuando `sonidoActual = 0`, `1`, etc., el microcontrolador necesita saber **dónde están guardados esos datos en la memoria flash** y **cuántas muestras dura cada uno**.

Un archivo de audio convertido a hexadecimal en C++ se ve típicamente así:

C++

```
const int16_t audio_golpe[] = { 0, 120, 450, -320, ... }; // Datos de las m
const uint32_t tamano_golpe = 12000;                      // Cantidad de mu
```

La duda es: **¿Cómo prefieres que la clase `Ampli` conozca esos datos?**

Opción A (Lista interna en la clase):
Crear una estructura simple dentro de la clase y una función para registrarlos al inicio. Por ejemplo:

C++

```
// En tu main:
miAmpli.agregarSonido(audio_golpe, tamano_golpe);   // Queda guardado
miAmpli.agregarSonido(audio_moneda, tamano_moneda); // Queda guardado

miAmpli.sonidoActual = 0; //[cite: 2]
miAmpli.reproducir();     // Reproduce audio_golpe[cite: 2]
```

Opción B (Arreglo externo):
Tener fuera de la clase un arreglo de estructuras con todos tus sonidos y pasarle a la clase ese arreglo global una sola vez al inicializarla.

Opción C (Pasar el sonido al llamar la función):
En lugar de configurar manualmente `sonidoActual` [cite: 2], llamar directamente `reproducir(0)` o pasar el sonido directamente, actualizando internamente la variable.

Dos detalles finales para confirmar:

1. **Resolución de las muestras:** En los arreglos a 11025 Hz, ¿las muestras son de **8 bits** (`uint8_t` / `int8_t` , típico de conversiones básicas) o de **16 bits** (`int16_t`)?
2. **Preferencia para el punto 3:** ¿Cuál de las opciones (A, B o C) se adapta mejor a cómo pensabas estructurar tus archivos de audio?

class Ampli conocerá el dato de manera referencial, la idea es tener un archivo .h o .cpp (aún investigando) que modele el funcionamiento del cambio de arreglo.
Es decir, que Audio.h o Audio.cpp (sujeto a cambic

Antes de generar el código definitivo, detecté dos detalles en tus archivos adjuntos que presentan errores de sintaxis en C++ o inconsistencias de nombres, por lo que requiero tu confirmación antes de corregirlos o eliminarlos:

1. **Declaración con inicialización inválida en `Ampli.h`:**
   En tu archivo adjunto aparece la línea:

   ```C++
   int pinLRC = int pinReloj + 1 ;
   ```

   En C++, dentro de la declaración de una clase no se puede escribir `int pinReloj + 1` de esa manera (da error de compilación). La forma estándar es calcular `pinLRC = pinReloj + 1` dentro del constructor en el `.cpp` o calcularlo al inicializar el bus. ¿Confirmas hacer ese cálculo en el constructor en lugar de la cabecera?

2. **Nombres de parámetros en el constructor de `Ampli.cpp`:**
   En tu constructor adjunto aparece:

   ```C++
   Ampli::Ampli(int pinReloj, int pinDatos) {
     pinReloj = nuevoPinReloj;
     pinDatos = nuevoPinDatos;
   ```

   Las variables `nuevoPinReloj` y `nuevoPinDatos` no están declaradas en los argumentos. ¿Deseas renombrar los argumentos a `Ampli::Ampli(int nuevoPinReloj, int nuevoPinDatos)` o asignar `this->pinReloj = pinReloj;`?

3. **Mecanismo asíncrono para no bloquear la ejecución:**
   Para reproducir sin congelar `controlVolumen()` ni el resto del microcontrolador, se requiere alimentar los buffers de audio continuamente. ¿Prefieres que agreguemos un método ligero `actualizar()` para invocar dentro de tu bucle principal `while(true)`, o prefieres que la clase lance automáticamente el proceso en el segundo núcleo (**Core 1**) de la Pico 2 W para que trabaje 100% de fondo sin tocar el bucle principal?

Apenas me confirmes estos tres puntos, procedo a generar los archivos siguiendo al pie de la letra las reglas de comentarios sin mayúsculas ni tildes.

1. Cambio la definición a `int pinLRC = pinReloj + 1`
2. Realiza el cambio por `Ampli::Ampli(int nuevoPinReloj, int nuevoPinDatos`
3. Profundiza en ambos puntos que propones, de

Respecto a tu primer punto, en C++ la declaración de miembros dentro del cuerpo de una clase no permite calcular un valor basándose en otro atributo al vuelo de esa manera sin inicializarlo en el constructor (o como valor estático). En el constructor de `Ampli` se asignará formalmente `pinReloj = nuevoPinReloj;` y de inmediato `pinLRC = pinReloj + 1;` . CPP +1

Análisis de los métodos de reproducción asíncrona

Para enviar audio al chip MAX98357A sin pausar el código ni congelar la lectura de variables como el volumen, el microcontrolador debe ir llenando pequeños bloques de memoria (búferes DMA) de manera constante.

Opción 1: Método `actualizar()` en el bucle principal (`Core 0`)

En este esquema, defines una función `miAmpli.actualizar()` que se coloca dentro del `while(true)` de tu archivo `main.cpp` . Cada vez que el bucle da una vuelta, el método verifica si la placa necesita más muestras de audio y, si es así, copia un bloque pequeño de datos sin detenerse a esperar.

Fortalezas:

Simplicidad de depuración: Todo el programa corre en un único flujo de instrucciones, facilitando el seguimiento paso a paso con mensajes por consola.

Sin conflictos de memoria: Al no compartir variables entre dos procesadores paralelos, se eliminan problemas de concurrencia o bloqueos de memoria (race conditions).

Bajo consumo de recursos: Deja el segundo núcleo totalmente libre para tareas futuras más complejas (sensores pesados, cálculos matemáticos o interfaces visuales).

Debilidades:

Sensible a funciones lentas: Si en tu bucle principal agregas un `sleep_ms()` largo o una lectura que tome mucho tiempo, el flujo de audio se interrumpirá y el parlante emitirá chasquidos (*buffer underrun*).

Requiere llamada manual: Depende de que recuerdes colocar `miAmpli.actualizar()` dentro de cada ciclo de tu código principal.

Opción 2: Segundo núcleo de la Pico (`Core 1`)

La Raspberry Pi Pico 2 W cuenta con dos núcleos independientes. Con la función `multicore_launch_core1()` , se puede iniciar una tarea dedicada en el segundo núcleo exclusivamente a alimentar el I2S de manera infinita.

Fortalezas:

Audio inmune a bloqueos: La reproducción nunca tartamudea ni sufre cortes, incluso si en el núcleo principal usas pausas de tiempo (`sleep_ms`) o procesos pesados.

Transparencia total en el `main.cpp` **:** Solo llamas a `reproducir()` y el segundo procesador se encarga de todo el trabajo de fondo de forma autónoma. `CPP + 1`

Debilidades:

Mayor complejidad de sincronización: Las variables que se modifican en un núcleo y se leen en el otro (como `volumen` o `sonidoActual`) requieren calificadores como `volatile` para evitar que el compilador las guarde en registros temporales. `H`

Uso completo de un procesador: Reserva el segundo núcleo exclusivamente para el sistema de sonido.

Recomendación

La recomendación para un proyecto modular de sintetizador o reproductor de samples en la Pico es la **Opción 2 (Core 1)**.

Al delegar el streaming de audio al segundo núcleo, puedes modificar `sonidoActual` , ajustar `volumen` con potenciómetros o ejecutar rutinas de control en el `Core 0` con total libertad de tiempos, garantizando que el chip MAX98357A reciba su flujo continuo de datos a 11025 Hz sin ruidos ni cortes de búfer. `H + 1`

¿Prefieres que implementemos la reproducción autónoma en el **Core 1**, o te inclinas por el método manual con `actualizar()` en el bucle principal?

Implementa la primera opción tomando como referencia todas directrices establecidas. Suma a estas, que no deben existir nada dentro private

A continuación tienes la implementación completa de `Ampli.h` y `Ampli.cpp` adaptada al chip MAX98357A, utilizando la primera opción (`actualizar()` sin bloqueo), con todos los miembros públicos, manteniendo los comentarios originales y sumando explicaciones en cada línea en minúsculas y sin tildes.

`Ampli.h`

C++

```cpp
#ifndef AMPLI_H
#define AMPLI_H

// incluye las herramientas basicas del sdk de raspberry pi
#include "pico/stdlib.h"
// incluye la libreria oficial de audio i2s por pio
#include "pico/audio_i2s.h"
// incluye tipos de datos enteros de tamano fijo como uint8_t y uint32_t
#include <stdint.h>
// incluye funciones matematicas para el calculo del volumen logaritmico
#include <math.h>

class Ampli {

  public:

  // --atributos--
  // a que pin se conecta el BCLK
  // Bit CLocK
  int pinReloj;

  // a que pin se conecta LRC
  // debe ir consecutivo a reloj
  int pinLRC;

  // a que pin se conecta el DIN
  // Serial Audio Data
  int pinDatos;

  bool sonando = false;
```

```
 int volumen = 50;
 int frecuencia = 11025;

// variable que enlaza al listado de sonidos que se carguen eventualmente
// cuenta desde 0
 int sonidoActual = -1;

 // guarda si la reproduccion actual debe repetirse en bucle
 bool enLoop = false;
 // almacena el puntero al arreglo de muestras de audio de 8 bits
 const uint8_t *datosAudio = nullptr;
 // guarda la cantidad total de muestras que tiene el sonido cargado
 uint32_t totalMuestras = 0;
 // lleva la cuenta de que muestra se esta enviando actualmente
 uint32_t posicionAudio = 0;
 // factor matematico de ganancia calculado de forma logaritmica
 float gananciaCalculada = 0.5f;

 // estructura que define las caracteristicas del formato de audio
 audio_format_t formatoAudio;
 // estructura que define como se empaqueta el buffer para el sistema dma
 audio_buffer_format_t formatoProductor;
 // configuracion de los pines fisicos y la maquina de estados pio
 struct audio_i2s_config configuracionI2S;
 // puntero a la cola de buffers de audio que maneja el chip
 struct audio_buffer_pool *colaBuffers = nullptr;

// -- constructor --
Ampli(int nuevoPinReloj, int nuevoPinDatos);

// --metodos--
// realiza el proceso de reproducir audio
// reproduce sonidoActual
void reproducir();

// el audio debe reproducirse de manera ciclica
//reproduce sonidoActual
void reproducirLoop();

// estipula como se regula el volumen
// utiliza valores de 0 a 100
void controlVolumen();

// entrega a la clase el arreglo de audio y su cantidad de muestras
void cargarAudio(const uint8_t *arregloMuestras, uint32_t cantidadMuestras

// alimenta el chip con muestras de sonido sin detener el resto del progra
void actualizar();
```

```
};

#endif
```

Ampli.cpp

C++

```
#include "Ampli.h"

//constructor
Ampli::Ampli(int nuevoPinReloj, int nuevoPinDatos) {

  // guarda el pin asignado para el reloj de bits
  pinReloj = nuevoPinReloj;
  // guarda el pin asignado para la linea de datos digitales
  pinDatos = nuevoPinDatos;
  // calcula de forma automatica el pin del reloj de canal sumando uno
  pinLRC = pinReloj + 1;

   // inicia el ampli MAX98357A pero sin reproducir ningún sonido
   //

  // define la velocidad de reproduccion en 11025 muestras por segundo
  formatoAudio.sample_freq = frecuencia;
  // define el formato de salida a 16 bits con signo que requiere el chip
  formatoAudio.format = AUDIO_BUFFER_FORMAT_PCM_S16;
  // define dos canales de salida para enviar la misma senal a izquierda y
  formatoAudio.channel_count = 2;

  // enlaza el formato base de audio con el productor
  formatoProductor.format = &formatoAudio;
  // indica que cada muestra combinada ocupa cuatro bytes en memoria
  formatoProductor.sample_stride = 4;

  // asigna el pin de datos din para la maquina pio
  configuracionI2S.data_pin = pinDatos;
  // asigna el pin base del reloj bclk para la maquina pio
  configuracionI2S.clock_pin_base = pinReloj;
  // selecciona el canal cero de transferencia directa dma
  configuracionI2S.dma_channel = 0;
  // selecciona la maquina de estados cero del pio
  configuracionI2S.pio_sm = 0;

  // crea una reserva de tres cajitas de memoria con capacidad de 256 muest
```

```
  colaBuffers = audio_new_producer_pool(&formatoProductor, 3, 256);

  // configura el hardware del microcontrolador con los parametros indicadc
  audio_i2s_setup(&formatoAudio, &configuracionI2S);
  // enlaza la reserva de memoria con la salida de sonido por pio
  audio_i2s_connect(colaBuffers);
  // enciende el flujo de senal digital hacia el amplificador
  audio_i2s_set_enabled(true);

  // calcula el nivel de volumen inicial para dejarlo listo
  controlVolumen();
}

// Inicia la reproducción de audio
// esta funciona con la variable sonidoActual
// tiene inicio y fin
void Ampli::reproducir(){
  // revisa si ya esta sonando para no reiniciar el audio si ya esta activc
  if (sonando) {
    // sale de la funcion sin hacer nada
    return;
  }
  // verifica que existan datos de audio validos para reproducir
  if (datosAudio == nullptr || totalMuestras == 0) {
    // sale de la funcion si no hay sonido cargado
    return;
  }
  // reinicia el puntero de lectura al inicio del sonido
  posicionAudio = 0;
  // indica que este modo no debe repetirse al terminar
  enLoop = false;
  // marca el estado como reproduciendo activamente
  sonando = true;
}

// A diferencia de reproducir()
// Inicia el sonido de modo ciclico
// funciona con la variable sonidoActual
void Ampli::reproducirLoop(){
  // revisa si ya esta sonando para evitar reiniciar la reproduccion
  if (sonando) {
    // sale de la funcion sin hacer nada
    return;
  }
  // verifica que existan datos de audio validos cargados
  if (datosAudio == nullptr || totalMuestras == 0) {
    // sale de la funcion si no hay sonido cargado
    return;
  }
```

```
  // reinicia el puntero de lectura al inicio del sonido
  posicionAudio = 0;
  // activa la bandera para que vuelva a empezar cuando llegue al final
  enLoop = true;
  // marca el estado como reproduciendo activamente
  sonando = true;
}

// Define como se controla el volumen
// se trabaja con valores de 0 a 100
void Ampli::controlVolumen(){
  // asegura que el volumen no sea menor que cero
  if (volumen < 0) {
    // fija el limite inferior en cero
    volumen = 0;
  }
  // asegura que el volumen no supere el valor de cien
  if (volumen > 100) {
    // fija el limite superior en cien
    volumen = 100;
  }

  // si el volumen es cero silencia por completo la salida
  if (volumen == 0) {
    // asigna cero a la ganancia para no emitir ruido
    gananciaCalculada = 0.0f;
  } else {
    // transforma el rango lineal de 0 a 100 a una escala logaritmica natur
    gananciaCalculada = (powf(10.0f, (float)volumen / 100.0f) - 1.0f) / 9.0
  }
}

// entrega a la clase el arreglo de audio y su cantidad de muestras
void Ampli::cargarAudio(const uint8_t *arregloMuestras, uint32_t cantidadMu
  // guarda la direccion en memoria del arreglo que contiene las muestras
  datosAudio = arregloMuestras;
  // guarda la cantidad de muestras que componen el sonido
  totalMuestras = cantidadMuestras;
  // reinicia el indice de lectura a la primera muestra
  posicionAudio = 0;
}

// alimenta el chip con muestras de sonido sin detener el resto del program
void Ampli::actualizar() {
  // intenta obtener una cajita de memoria libre sin detener el programa si
  struct audio_buffer *buffer = take_audio_buffer(colaBuffers, false);

  // si no hay cajitas de memoria listas sale de inmediato para no congelar
  if (buffer == nullptr) {
```

```
  // regresa al programa principal
  return;
}

// apunta al espacio de memoria donde se guardaran las muestras calculada
int16_t *muestrasSalida = (int16_t *)buffer->buffer->bytes;
// determina la cantidad de muestras que caben en esta cajita
uint16_t espacioDisponible = buffer->max_sample_count;

// recorre cada espacio disponible dentro de la cajita de memoria
for (uint16_t i = 0; i < espacioDisponible; i++) {
  // variable temporal para guardar el valor de la muestra
  int16_t muestraProcesada = 0;

  // revisa si el sonido esta activo y hay datos por leer
  if (sonando && datosAudio != nullptr) {
    // toma la muestra de 8 bits sin signo de 0 a 255
    uint8_t muestraOriginal = datosAudio[posicionAudio];
    // centra el audio restando 128 para convertirlo a rango con signo de
    int16_t muestraCentrada = (int16_t)muestraOriginal - 128;
    // amplifica la senal a 16 bits multiplicando y aplicando la atenuaci
    muestraProcesada = (int16_t)((float)muestraCentrada * 256.0f * gananc

    // avanza a la siguiente muestra del arreglo
    posicionAudio++;

    // revisa si se llego al final de todas las muestras del sonido
    if (posicionAudio >= totalMuestras) {
      // si esta configurado en bucle vuelve al comienzo del sonido
      if (enLoop) {
        // regresa el puntero al inicio del audio
        posicionAudio = 0;
      } else {
        // detiene la reproduccion al finalizar
        sonando = false;
        // regresa el puntero al inicio para la proxima vez
        posicionAudio = 0;
      }
    }
  } else {
    // si no esta sonando envia silencio absoluto
    muestraProcesada = 0;
  }

  // envia la muestra calculada al canal de sonido izquierdo
  muestrasSalida[i * 2] = muestraProcesada;
  // envia la misma muestra al canal derecho para tener sonido en ambos l
  muestrasSalida[i * 2 + 1] = muestraProcesada;
}
```

```
    // indica cuantas muestras fueron escritas en la cajita de memoria
    buffer->sample_count = espacioDisponible;
    // entrega la cajita llena al sistema dma para que la envie al chip de sc
    give_audio_buffer(colaBuffers, buffer);
}
```

Entrega un informe que indique que atributos y métodos se agregaron en .h y .cpp. Además de explicar cómo debería utilizarse esta clase por ejemplo al presionar un botón

Informe de Modificaciones: Clase `Ampli`

1. Elementos Agregados en `Ampli.h`

Respecto al archivo base original, se conservaron todos los miembros públicos iniciales (`pinReloj` , `pinLRC` , `pinDatos` , `sonando` , `volumen` , `frecuencia` , `sonidoActual` , `reproducir` , `reproducirLoop` y `controlVolumen`) y se incorporaron los siguientes atributos y métodos dentro de la sección pública: `H`

Nuevos Atributos

`bool enLoop` : Bandera booleana que determina si la reproducción debe reiniciar desde la muestra cero al llegar al final del arreglo.

`const uint8_t *datosAudio` : Puntero a la memoria flash donde reside el arreglo de bytes hexadecimales con las muestras PCM de 8 bits.

`uint32_t totalMuestras` : Almacena el número total de muestras que componen el sonido en curso.

`uint32_t posicionAudio` : Índice de lectura que rastrea la muestra actual que está siendo procesada.

`float gananciaCalculada` : Escala multiplicativa calculada a partir de `volumen` mediante la fórmula logarítmica para atenuar las muestras de audio. `H`

`audio_format_t formatoAudio` : Estructura del SDK de la Pico que define la frecuencia (11025 Hz), el formato PCM de 16 bits y los dos canales de audio.

`audio_buffer_format_t formatoProductor` : Estructura que especifica el tamaño de paso en bytes (`sample_stride = 4`) para el empaquetado del búfer DMA.

`struct audio_i2s_config configuracionI2S` : Configuración de los periféricos de hardware (pines GPIO, canal DMA y máquina de estados PIO).

`struct audio_buffer_pool *colaBuffers` : Puntero a la cola de búferes circular gestionada por el SDK.

Nuevos Métodos

`void cargarAudio(const uint8_t *arregloMuestras, uint32_t cantidadMuestras)` : Vincula el arreglo de muestras y su longitud con las variables internas de la clase.

`void actualizar()` : Función de procesamiento no bloqueante. Revisa si la cola DMA requiere más datos, escala las muestras de 8 bits a 16 bits con el volumen actual y las envía al bus I2S.

2. Elementos Implementados en `Ampli.cpp`

En la unidad de implementación se codificaron las rutinas completas que antes estaban vacías: CPP

Constructor `Ampli::Ampli(int nuevoPinReloj, int nuevoPinDatos)` :

Asignación de `pinReloj` , `pinDatos` y cálculo automático de `pinLRC = pinReloj + 1` .

Inicialización de las estructuras I2S del SDK (`audio_i2s_setup` , `audio_i2s_connect` , `audio_i2s_set_enabled`).

Llamada inicial a `controlVolumen()` para precalcular la ganancia. CPP

`void reproducir()` :

Comprueba si el sonido ya está activo (`if (sonando) return;`) para evitar reentradas que interrumpan el audio. H

Valida la existencia de datos y fija `enLoop = false` .

`void reproducirLoop()` :

Misma lógica de guarda anti-reinicio, configurando `enLoop = true` para reproducción cíclica. CPP + 1

`void controlVolumen()` :

Acota el valor de `volumen` al rango $[0, 100]$. H

Aplica la transformación logarítmica:

$$\text{gananciaCalculada} = \frac{10^{\frac{\text{volumen}}{100}} - 1}{9}$$

`void cargarAudio(...)` :

Inicializa el puntero de muestras y resetea el índice `posicionAudio = 0` .

`void actualizar()` :

Adquiere un búfer libre usando `take_audio_buffer(..., false)` (modo no bloqueante).

Convierte las muestras mono de 8 bits sin signo $(0 \ldots 255)$ a 16 bits con signo $(-32768 \ldots 32767)$ mediante la operación `(muestra - 128) * 256 * gananciaCalculada` .

Duplica cada muestra mono en los canales izquierdo y derecho (L/R) y entrega el bloque al DMA mediante `give_audio_buffer()` .

3. Ejemplo de Uso con un Botón en `main.cpp`

El siguiente ejemplo muestra la integración de la clase `Ampli` con un botón físico conectado al **GPIO 15** (con resistencia `pull-up` interna activa, cerrando a tierra al presionar).

El código utiliza el modelo no bloqueante: la función `actualizar()` se invoca continuamente dentro del ciclo principal, permitiendo detectar la pulsación del botón y disparar el sonido sin demoras ni congelamiento.

C++

```cpp
#include "pico/stdlib.h"
#include "Ampli.h"

// pin asignado para el pulsador
#define PIN_BOTON 15

// pines del max98357a
#define PIN_BCLK 26 // reloj de bit
#define PIN_DIN  28 // datos de audio (lrc queda asignado automaticamente e

// ejemplo de arreglo con muestras pcm mono de 8 bits a 11025 hz
const uint8_t sonidoMuestra[] = {
    128, 140, 160, 185, 210, 230, 245, 255,
    245, 230, 210, 185, 160, 140, 128, 115,
    95,  70,  45,  25,  10,  0,   10,  25,
    45,  70,  95,  115, 128
};
const uint32_t tamanoSonido = sizeof(sonidoMuestra);
```

```
int main() {
    stdio_init_all();

    // 1. inicializar pin del boton con pull-up interno
    gpio_init(PIN_BOTON);
    gpio_set_dir(PIN_BOTON, GPIO_IN);
    gpio_pull_up(PIN_BOTON);

    // 2. instanciar la clase ampli
    Ampli miAmpli(PIN_BCLK, PIN_DIN);

    // 3. cargar el sonido a reproducir
    miAmpli.cargarAudio(sonidoMuestra, tamanoSonido);

    // 4. configurar el nivel de volumen inicial (0 a 100)
    miAmpli.volumen = 70;
    miAmpli.controlVolumen();

    // variables para detectar flanco de bajada (evitar lecturas falsas al
    bool estadoAnteriorBoton = true;

    while (true) {
        // tarea prioritaria: alimentar el amplificador con muestras contir
        miAmpli.actualizar();

        // lectura digital del estado del pulsador
        bool estadoActualBoton = gpio_get(PIN_BOTON);

        // deteccion de pulsacion: pasa de alto (1) a bajo (0)
        if (estadoAnteriorBoton == true && estadoActualBoton == false) {
            // reproduce el sonido asignado (si ya esta sonando, la llamada
            miAmpli.reproducir();
        }

        estadoAnteriorBoton = estadoActualBoton;

        // pequena pausa de desrebote mecanico (debouncing)
        sleep_ms(5);
    }

    return 0;
}
```

↝ Desde Audio I2S con MAX98357A y Pico 2 W